■ SQL
Interview Questions & Answers

For Freshers — Simple English, Easy to Remember

60+ Most-Asked SQL Questions with Clear Answers + Real Examples

SECTION 1 — SQL Basics

■ What is SQL?

Q1. What is SQL?

Answer: SQL stands for Structured Query Language. It is used to talk to a database — to store, get, update, and delete data. Think of a database like a big Excel file with many sheets (called tables), and SQL is how you ask questions to that file.

*Example: SELECT * FROM students; — this gets all data from the students table.*

Q2. What are the different parts (sub-languages) of SQL?

Answer: SQL has 4 parts: (1) DDL — to CREATE, ALTER, DROP tables. (2) DML — to SELECT, INSERT, UPDATE, DELETE data. (3) DCL — to GRANT or REVOKE permissions. (4) TCL — to COMMIT or ROLLBACK transactions.

Remember: DDL=structure, DML=data, DCL=permissions, TCL=transactions

Q3. What is a Table in SQL?

Answer: A table is like a spreadsheet with rows and columns. Each row is one record (like one student's data). Each column is one attribute (like Name, Age, Marks).

Example: Table 'Students' has columns: StudentID, Name, Age, Marks.

Q4. What is a Primary Key?

Answer: A primary key is a column (or group of columns) that uniquely identifies each row. No two rows can have the same primary key value. It also cannot be empty (NULL).

Example: StudentID is the primary key — every student has a unique ID.

Remember: Primary Key = Unique + Not Null

Q5. What is a Foreign Key?

Answer: A foreign key is a column in one table that points to the primary key of another table. It creates a link between two tables.

Example: Orders table has CustomerID column that points to CustomerID in the Customers table.

Q6. What is the difference between NULL and 0 or empty string?

Answer: NULL means 'no value / unknown'. 0 is a number. Empty string '' is text with nothing in it. NULL is not equal to anything — even NULL != NULL in SQL. Use IS NULL or IS NOT NULL to check for NULL.

Example: WHERE salary IS NULL finds rows where salary is missing.

Q7. What is a Database? What is a Schema?

Answer: A Database is a container that holds all your tables and data. A Schema is the structure/design of the database — which tables exist, what columns they have, what data types they use.

Remember: Database = building, Schema = blueprint of the building

■ SELECT Queries

Q8. How do you get data from a table?

Answer: Use SELECT. To get everything: SELECT * FROM table_name; To get specific columns: SELECT name, age FROM students;

Example: SELECT name, marks FROM students; — gets only name and marks columns.

Q9. How do you filter rows using conditions?

Answer: Use WHERE clause. You can use =, >, <, >=, <=, != operators.

*Example: SELECT * FROM students WHERE marks > 80; — gets students who scored more than 80.*

Q10. How do you sort results?

Answer: Use ORDER BY. ASC = smallest to biggest (default). DESC = biggest to smallest.

*Example: SELECT * FROM students ORDER BY marks DESC; — shows highest marks first.*

Q11. How do you limit the number of results shown?

Answer: Use LIMIT (MySQL/PostgreSQL) or TOP (SQL Server).

*Example: SELECT * FROM students LIMIT 5; — shows only first 5 rows.*

Q12. What does DISTINCT do?

Answer: DISTINCT removes duplicate values and shows only unique values.

Example: SELECT DISTINCT city FROM customers; — shows each city only once.

Q13. What is the difference between WHERE and HAVING?

Answer: WHERE filters rows BEFORE grouping. HAVING filters AFTER grouping. Simple rule: Use WHERE for row conditions, use HAVING for group conditions.

Example: SELECT city, COUNT(*) FROM orders GROUP BY city HAVING COUNT(*) > 10;

Remember: WHERE = rows, HAVING = groups

Q14. What is LIKE operator?

Answer: LIKE is used for pattern matching. % means 'any number of characters'. _ means 'exactly one character'.

*Example: SELECT * FROM students WHERE name LIKE 'A%'; — names starting with A.*

Q15. What is the IN operator?

Answer: IN is used to match a value against a list of values. It's a shortcut for multiple OR conditions.

*Example: SELECT * FROM students WHERE city IN ('Mumbai','Pune','Delhi');*

Q16. What is BETWEEN?

Answer: BETWEEN filters rows where a value falls within a range (inclusive on both ends).

*Example: SELECT * FROM products WHERE price BETWEEN 100 AND 500;*

SECTION 2 — JOINs

Q17. What is a JOIN? Why do we use it?

Answer: A JOIN combines rows from two or more tables based on a related column. We use it because data is stored in separate tables and sometimes we need information from multiple tables at once.

Example: Combine Students table and Marks table using StudentID.

Q18. What is INNER JOIN?

Answer: INNER JOIN returns only the rows that have matching values in BOTH tables. If a row in Table A has no match in Table B, it is NOT included.

Example: SELECT s.name, m.marks FROM students s INNER JOIN marks m ON s.id = m.student_id;

Remember: INNER JOIN = only matching rows

Q19. What is LEFT JOIN (LEFT OUTER JOIN)?

Answer: LEFT JOIN returns ALL rows from the LEFT table, and matching rows from the right table. If there is no match in the right table, NULL is shown.

Example: SELECT s.name, o.order_date FROM students s LEFT JOIN orders o ON s.id = o.student_id; — shows all students even if they have no orders.

Remember: LEFT JOIN = all from left + matched from right

Q20. What is RIGHT JOIN?

Answer: RIGHT JOIN returns ALL rows from the RIGHT table and matching rows from the left table. It is the opposite of LEFT JOIN.

Remember: RIGHT JOIN = all from right + matched from left

Q21. What is FULL OUTER JOIN?

Answer: FULL OUTER JOIN returns ALL rows from BOTH tables. Where there is no match, NULL is shown on the missing side.

Remember: FULL OUTER JOIN = everything from both tables

Q22. What is CROSS JOIN?

Answer: CROSS JOIN returns every combination of rows from both tables. If Table A has 3 rows and Table B has 4 rows, result has 3x4=12 rows. Use carefully — it can produce huge results.

Q23. What is SELF JOIN?

Answer: SELF JOIN is when a table joins with itself. Useful when a table has a column that references another row in the same table.

Example: Employees table has ManagerID column. Self join to find each employee's manager name.

Q24. How do you find duplicate rows in a table?

Answer: Use GROUP BY on the column and HAVING COUNT(*) > 1.

Example: SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1; — finds duplicate emails.

SECTION 3 — Aggregate & Group Functions

Q25. What are Aggregate Functions?

Answer: Aggregate functions work on a group of rows and return a single value. Common ones: COUNT() — count rows, SUM() — total, AVG() — average, MIN() — smallest, MAX() — biggest.

Example: SELECT AVG(salary) FROM employees; — gives average salary.

Q26. What does GROUP BY do?

Answer: GROUP BY groups rows that have the same value in a column, then you can apply aggregate functions on each group.

Example: SELECT department, AVG(salary) FROM employees GROUP BY department; — average salary per department.

Q27. What is COUNT(*) vs COUNT(column)?

Answer: COUNT(*) counts all rows including NULLs. COUNT(column) counts only non-NULL values in that column.

Example: If 10 rows exist but 3 have NULL salary: COUNT(*) = 10, COUNT(salary) = 7.

Q28. What is the difference between SUM and COUNT?

Answer: COUNT counts how many rows. SUM adds up the values. COUNT(orders) = how many orders. SUM(order_amount) = total money.

SECTION 4 — Window Functions

Q29. What is a Window Function?

Answer: A window function does a calculation across a set of rows related to the current row — WITHOUT collapsing rows into one (unlike GROUP BY). Think of it as 'calculate something for each row while looking at surrounding rows'.

Example: ROW_NUMBER() OVER (ORDER BY salary DESC) — gives rank to each row.

Remember: Window functions keep all rows; GROUP BY reduces rows.

Q30. What is ROW_NUMBER()?

Answer: ROW_NUMBER() gives a unique sequential number to each row. Even if two rows have the same value, they get different numbers. Used for pagination or getting top N rows.

Example: ROW_NUMBER() OVER (ORDER BY marks DESC) — rank 1,2,3,4...

Q31. What is RANK()?

Answer: RANK() gives the same rank to rows with the same value, but then skips the next number. Like sports ranking: two students both score 90, both get rank 1, next student gets rank 3 (rank 2 is skipped).

Example: Scores 90,90,80 → RANK: 1,1,3

Remember: RANK skips numbers after a tie.

Q32. What is DENSE_RANK()?

Answer: DENSE_RANK() is like RANK() but does NOT skip numbers. Two students both score 90, both get rank 1, next student gets rank 2 (no gap).

Example: Scores 90,90,80 → DENSE_RANK: 1,1,2

Remember: DENSE_RANK = no gaps, RANK = has gaps

Q33. What is PARTITION BY?

Answer: PARTITION BY divides rows into groups (like GROUP BY) but for window functions. The window function restarts for each partition.

Example: RANK() OVER (PARTITION BY department ORDER BY salary DESC) — rank employees within each department separately.

Q34. What is LAG() and LEAD()?

Answer: LAG(column, n) gets the value from n rows BEFORE the current row. LEAD(column, n) gets the value from n rows AFTER the current row. Very useful for month-over-month comparisons.

Example: LAG(sales,1) OVER (ORDER BY month) — gets last month's sales for comparison.

Q35. How do you find the 2nd highest salary?

Answer: Method 1 - Subquery: SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees); Method 2 - DENSE_RANK: SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees) t WHERE rnk = 2;

Remember: Learn both methods — very commonly asked!

SECTION 5 — Advanced SQL

Q36. What is a Subquery?

Answer: A subquery is a query inside another query. The inner query runs first and its result is used by the outer query.

Example: SELECT name FROM students WHERE marks > (SELECT AVG(marks) FROM students); — finds students above average.

Q37. What is a CTE (Common Table Expression)?

Answer: A CTE is like a temporary table you define at the top of a query using WITH keyword. It makes long queries easier to read.

*Example: WITH top_students AS (SELECT * FROM students WHERE marks > 90) SELECT * FROM top_students;*

Remember: WITH clause = CTE = temporary named result

Q38. What is the difference between DELETE, TRUNCATE, and DROP?

Answer: DELETE removes specific rows (you can add WHERE). You can undo it with ROLLBACK. TRUNCATE removes ALL rows at once — faster but cannot be undone. DROP removes the ENTIRE TABLE including structure. Cannot be undone.

Remember: DELETE=rows, TRUNCATE=all rows, DROP=whole table

Q39. What is UNION vs UNION ALL?

Answer: UNION combines results of two queries and removes duplicate rows. UNION ALL combines and keeps ALL rows including duplicates. UNION ALL is faster because it skips the duplicate-removal step.

Example: Both queries must have same number of columns with compatible data types.

Q40. What is an Index?

Answer: An index is like the index at the back of a book — it helps the database find data faster without reading every row. It speeds up SELECT queries but slows down INSERT/UPDATE/DELETE.

Example: CREATE INDEX idx_name ON students(name); — creates index on name column.

Q41. What are Stored Procedures?

Answer: A stored procedure is a saved SQL code that you can run again and again just by calling its name. Like a function/method in programming.

*Example: CREATE PROCEDURE GetTopStudents AS SELECT * FROM students WHERE marks > 90;*

Q42. What is a View?

Answer: A view is a virtual table created from a SELECT query. It does not store data — it just saves the query. Every time you query the view, it runs the saved query.

*Example: CREATE VIEW high_earners AS SELECT * FROM employees WHERE salary > 50000;*

Q43. What are ACID properties? (Simple version)

Answer: ACID ensures database transactions are reliable. A = Atomicity (all steps complete or none do). C = Consistency (data stays valid). I = Isolation (transactions don't interfere). D = Durability (committed data is saved permanently even after crash).

Remember: Think: bank transfer — either both debit+credit happen, or neither.

Q44. What is Normalization? (Simple version)

Answer: Normalization is organizing a database to reduce repeated data. 1NF: each cell has one value only. 2NF: every column depends on the full primary key. 3NF: no column depends on another non-key column.

Remember: Goal: reduce duplicate data, save storage, avoid errors.

Q45. What is the difference between CHAR and VARCHAR?

Answer: CHAR stores fixed-length text — always uses the same space. VARCHAR stores variable-length text — uses only as much space as needed. CHAR('Hello') in CHAR(10) uses 10 characters. VARCHAR uses only 5.

Remember: Use CHAR for fixed-size things like gender (M/F), country codes. Use VARCHAR for names, addresses.

Q46. What is a Transaction? What are COMMIT and ROLLBACK?

Answer: A transaction is a group of SQL operations treated as one unit. COMMIT saves all the changes permanently. ROLLBACK undoes all changes if something goes wrong.

Example: Bank transfer: deduct from A, add to B — if adding fails, ROLLBACK undoes the deduction too.

Quick Reference Cheat Sheet

Command	What it does	Simple Example
SELECT	Get data	SELECT * FROM table
WHERE	Filter rows	WHERE age > 18
ORDER BY	Sort results	ORDER BY salary DESC
GROUP BY	Group rows	GROUP BY department
HAVING	Filter groups	HAVING COUNT(*) > 5
INNER JOIN	Only matching rows	JOIN b ON a.id = b.id
LEFT JOIN	All from left	LEFT JOIN ...
COUNT()	Count rows	COUNT(*) or COUNT(col)
SUM()	Add values	SUM(salary)
AVG()	Average	AVG(marks)
ROW_NUMBER()	Unique row number	ROW_NUMBER() OVER (ORDER BY sal)
RANK()	Rank with gaps	RANK() OVER (ORDER BY sal)
DENSE_RANK()	Rank no gaps	DENSE_RANK() OVER (ORDER BY sal)
LAG()	Previous row value	LAG(sales,1) OVER (ORDER BY month)
DISTINCT	Unique values	SELECT DISTINCT city
LIKE	Pattern search	name LIKE 'A%'
IN	Match list	city IN ('Mumbai','Delhi')
IS NULL	Check empty	WHERE phone IS NULL
COALESCE	Replace NULL	COALESCE(phone,'N/A')

All the Best! Practice writing queries daily.